AI Website Fabricator

Complete End-to-End Code Flow Document

Jury-Ready Technical Reference with File References

This document provides a **complete, end-to-end technical walkthrough** of the AI Website Fabricator project. It covers every module, algorithm, database schema, and data flow with exact file references. The project is a **Streamlit + Flask hybrid system** that clones any website, analyzes its structure, auto-generates a database schema and admin panel, and packages everything as a standalone deployable system.

Project File Map

File	Size	Role
app.py	~99 KB	Main Streamlit UI, auth, database, orchestration
cloner_engine.py	~74 KB	Website crawling, asset download, HTML rewriting
smart_conversion.py	~69 KB	Site analysis, model inference, blueprint generation
connected_export.py	~50 KB	Standalone export packaging with Flask backend
dragdrop_flask.py	~50 KB	Flask server, GrapesJS editor, admin API
admin/src/ (React+TS)	~20 files	Admin panel SPA (React 19, TypeScript, Vite)
users.db	SQLite	All user data, sites, models, records, media

Table of Contents

1. System Architecture Overview
2. Application Startup Flow
3. Authentication System
4. Database Schema (All 8 Tables)
5. Module 1: Website Cloning Engine (cloner_engine.py)
6. Module 2: Smart Dynamic Conversion (smart_conversion.py)
7. Module 3: Main Application Orchestrator (app.py)
8. Module 4: Drag & Drop Visual Editor (dragdrop_flask.py)
9. Module 5: Connected Export Packager (connected_export.py)
10. Module 6: Admin Panel Frontend (admin/src/)
11. End-to-End User Journey Flow
12. API Endpoints Reference
13. Key Algorithms Deep Dive
14. Diagram Explanations

1. System Architecture Overview

The system is a **hybrid Streamlit + Flask architecture**. Streamlit (app.py) serves the main UI on port 8501. Flask (dragdrop_flask.py) runs on port 5001 for the visual editor and admin API. Both share the same SQLite database (users.db). The exported system runs standalone Flask on port 5010.

Architecture Diagram

Diagram: The user interacts with Streamlit. Streamlit calls cloner_engine and smart_conversion modules. Flask provides the drag-drop editor (embedded as iframe) and the admin API. connected_export packages everything into a standalone ZIP with its own Flask server.

Module Dependency Graph

Module	Imports From	Purpose of Import
app.py	cloner_engine	clone_site_bundle, persist_bundle, load_bundle, normalize_url
app.py	smart_conversion	build_conversion_blueprint, build_site_spec, create_scaffold_files, inject_clone_bridge
app.py	connected_export	build_connected_export (packaging)
dragdrop_flask.py	cloner_engine	get_saved_bundle_site_dir, load/update_saved_bundle_entry
dragdrop_flask.py	smart_conversion	inject_clone_bridge
connected_export.py	cloner_engine	get_saved_bundle_site_dir, load_bundle_manifest, strip_preview_paths
connected_export.py	smart_conversion	inject_clone_bridge

Every module dependency with the specific functions imported.

2. Application Startup Flow

File: app.py (bottom of file), dragdrop_flask.py (bottom of file)

The system starts in two stages:

- **Stage 1 - Streamlit**: User runs `streamlit run app.py`. This initializes the database via `init_db()` which creates all 8 tables if they don't exist. Then it checks session state and shows either the login page or the main app.
- **Stage 2 - Flask**: When the user navigates to 'Drag & Drop Builder', `ensure_flask_running()` checks if port 5001 is active via `socket.connect_ex()`. If not, it spawns `dragdrop_flask.py` as a subprocess.

File: app.py : init_db() - lines ~50-120

```
def init_db():
  conn = sqlite3.connect(DB_NAME) # 'users.db'
  c = conn.cursor()
  c.execute('CREATE TABLE IF NOT EXISTS users ...')
  c.execute('CREATE TABLE IF NOT EXISTS cloned_sites ...')
  # ... creates all 8 tables
  conn.commit(); conn.close()
```

File: app.py : ensure_flask_running()

```
def ensure_flask_running():
  if not flask_running(FLASK_PORT): # socket check port 5001
    subprocess.Popen([sys.executable, 'dragdrop_flask.py'])
```

3. Authentication System

3.1 Streamlit Auth (Main App)

File: app.py : hash_password(), login_user(), create_user()

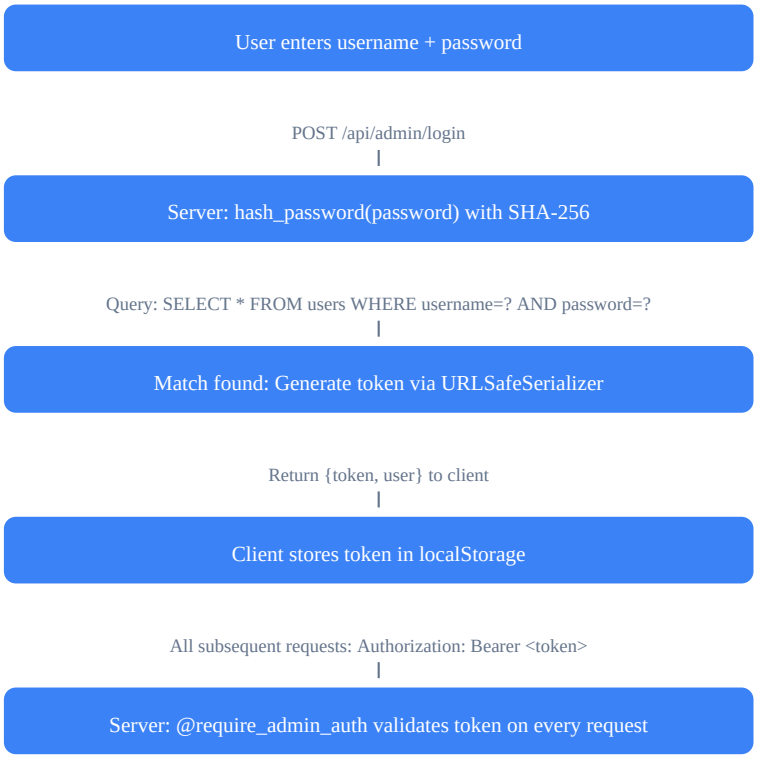
Passwords are hashed with **SHA-256** via `hashlib.sha256(password.encode()).hexdigest()`. Login queries the `users` table matching username AND password hash. On success, `st.session_state['logged_in'] = True` and user tuple is stored in `st.session_state['user_info']` = (id, username, hash, created_at).

3.2 Flask Admin Auth (API)

File: dragdrop_flask.py : /api/admin/login, require_admin_auth decorator

The Flask admin API uses **token-based auth** via `itsdangerous.URLSafeSerializer`. On POST to `/api/admin/login`, credentials are verified against the database. If valid, a token is generated: `serializer.dumps({user_id, username})`. All protected endpoints use `@require_admin_auth` which extracts the Bearer token from the Authorization header and validates it.

Auth Flow Diagram



Auth flow: Login generates a serialized token. All admin API calls require this token in the Authorization header.

4. Database Schema (SQLite - users.db)

`File: app.py : init_db() | dragdrop_flask.py : ensure_dynamic_tables()`

The database has **8 tables**. The first 3 are created in app.py's init_db(). The remaining 5 are created/migrated by dragdrop_flask.py's ensure_dynamic_tables(). All tables share user_id and site_id as foreign keys for multi-user isolation.

4.1 users

Column	Type	Constraint	Purpose
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique user ID
username	TEXT	UNIQUE NOT NULL	Login username
password	TEXT	NOT NULL	SHA-256 hashed password
created_at	TEXT		Registration timestamp

4.2 cloned_sites

Column	Type	Purpose
id	INTEGER PK	Unique site ID
user_id	INTEGER FK->users.id	Owner
url	TEXT	Original website URL that was cloned
html_content	TEXT	Full cloned HTML with bridge injected
cloned_at	TEXT	Clone timestamp

4.3 conversion_blueprints

Column	Type	Purpose
id	INTEGER PK	Blueprint ID
site_id	INTEGER UNIQUE FK	One blueprint per site
user_id	INTEGER FK	Owner
site_type	TEXT	Detected type: blog, ecommerce, portfolio, etc.
confidence	REAL	Detection confidence 0.0-1.0
blueprint_json	TEXT	Full blueprint (models, fields, routes, bindings)
generated_at	TEXT	Generation timestamp
artifact_dir	TEXT	Path to generated scaffold files

4.4 site_specs

Column	Type	Purpose
id	INTEGER PK	Spec ID
site_id	INTEGER UNIQUE FK	One spec per site
user_id	INTEGER FK	Owner
spec_json	TEXT	Full SiteSpec: projectId, resources, bindings
bindings_json	TEXT	Frontend data bindings array (selector -> API mapping)
updated_at	TEXT	Last update timestamp

4.5 dynamic_models

Column	Type	Purpose
id	INTEGER PK	Model ID
site_id	INTEGER FK	Parent site
user_id	INTEGER FK	Owner
model_name	TEXT	e.g., 'products', 'posts', 'courses'
section_title	TEXT	Admin panel section label
schema_json	TEXT	Field definitions: [{name, type, label, required}]
display_order	INTEGER	UI ordering in admin sidebar
updated_at	TEXT	Last update

UNIQUE constraint on (site_id, model_name) prevents duplicate models per site.

4.6 dynamic_records

Column	Type	Purpose
id	INTEGER PK	Record ID
site_id	INTEGER FK	Parent site
user_id	INTEGER FK	Owner
model_name	TEXT	References dynamic_models.model_name
data_json	TEXT	Record payload as arbitrary JSON
slug	TEXT	URL-friendly identifier (auto-generated)
status	TEXT DEFAULT 'draft'	Workflow state: 'draft' or 'published'
created_at / updated_at	TEXT	Timestamps

4.7 media_assets

Column	Type	Purpose
id	INTEGER PK	Asset ID
site_id, user_id	INTEGER FKs	Owner references
asset_key	TEXT	Unique identifier (content hash)
source_kind	TEXT	'upload' or 'clone'
media_type	TEXT	'image', 'svg', 'icon', 'video', 'audio'
mime_type	TEXT	e.g., 'image/png'
filename / original_name	TEXT	Local and original filenames

Column	Type	Purpose
file_url / original_url	TEXT	Serve path and source URL
alt_text	TEXT	Image accessibility text
width, height	INTEGER	Pixel dimensions
hash	TEXT	SHA-1 content hash for deduplication
status	TEXT	'ready', 'custom', 'deleted', 'archived'
usage_json	TEXT	Array of usage contexts [{selector, section}]
metadata_json	TEXT	Extra metadata as JSON

Indexed: idx_media_assets_site_user_asset_key on (site_id, user_id, asset_key).

5. Module 1: Website Cloning Engine

File: cloner_engine.py (~74 KB, ~1800 lines)

This module performs the actual website cloning. It crawls pages via BFS, downloads all assets (CSS, JS, images, fonts, video, audio), rewrites URLs to local paths, and packages everything into a bundle.

5.1 Cloning Algorithm (6 Phases)

Phase 1: Initialization

File: cloner_engine.py : clone_site_bundle()

Entry point is `clone_site_bundle(root_url, base_dir, max_pages, max_depth, progress_callback)`. It validates the URL, creates a temp staging directory at `project_assets/staged_clones/{domain}-{timestamp}-{hash}/`, and initializes a `requests.Session` with Chrome user-agent, connection pooling (12 workers), and keep-alive.

Phase 2: Page Discovery & Crawl (BFS)

File: cloner_engine.py : crawl_site_pages(), collect_internal_navigation_urls()

Uses **Breadth-First Search** with `collections.deque`. Steps: (a) Check `/robots.txt` and `/sitemap.xml` for seed URLs. (b) For each page, fetch HTML with 15s timeout. (c) Extract links from: `<a href>`, `data-href`, `onclick` handlers (3 regex patterns for window.location), `<link rel='next/prev'>`, `<meta refresh>`. (d) Deduplicate and enqueue if within max_pages and max_depth. Skips: non-HTML extensions, wp-admin, wp-json, feed, xmlrpc paths.

Phase 3: Asset Collection & Parallel Prefetch

File: cloner_engine.py : collect_page_asset_requests(), prefetch_assets()

For each crawled page, scans for assets via `iter_asset_attributes()` which checks: src, data-src, data-original, data-lazy-src, data-bg, poster, srcset, and any attribute containing 'src|image|lazy|background|thumb'. Detects asset kind (css/js/image/font/video/audio/icon). Then `prefetch_assets()` downloads all assets in parallel using `ThreadPoolExecutor(max_workers=12)`.

Phase 4: HTML Rewriting

File: cloner_engine.py : rewrite_page_html()

For each page: (a) Removes <base> tags, CSP meta tags, integrity attributes. (b) Rewrites ALL URLs in link href, img src, script src, inline styles url(), srcset, meta refresh. (c) Converts absolute URLs to relative local paths. (d) Creates placeholder pages for unreachable internal links. (e) Injects a **navigation bridge script** that handles dropdown toggles, data-href clicks, and keyboard navigation.

Phase 5: Asset Download & CSS Rewriting

File: cloner_engine.py : clone_asset(), clone_binary_asset(), clone_stylesheet_asset()

Binary assets (images, fonts, JS) are written to output with SHA-1 hash deduplication. CSS files get special treatment: `rewrite_css_text()` processes all `@import` and `url()` references, converting absolute URLs to relative paths. Hash index prevents writing the same file twice.

Phase 6: Bundling & Export

File: cloner_engine.py : clone_site_bundle() (end section)

Writes all rewritten pages to `site/` directory. Creates `manifest.json` with source_url, page/asset counts, route_map (source->local), and full diagnostics. Generates `clone-export.zip` archive. Returns result dict with entry_html, page_count, asset_count, bundle_dir, archive_path.

5.2 Output File Structure

```
project_assets/staged_clones/{domain}-{timestamp}-{hash}/
  site/
    index.html            # Root page
    pages/{slug}-{hash}.html  # Deep-nested pages
    assets/
      css/{name}-{hash}.css
      js/{name}-{hash}.js
```

```
    images/{name}-{hash}.png
    fonts/{name}-{hash}.woff2
    video/{name}-{hash}.mp4
    audio/{name}-{hash}.mp3
    icons/{name}-{hash}.ico
manifest.json
clone-export.zip
```

Each asset name is slugified (max 24 chars) + 8-char SHA-1 hash for uniqueness.

5.3 Key Constants

Constant	Value	Purpose
PAGE_REQUEST_TIMEOUT	15 seconds	HTTP timeout for page fetches
ASSET_REQUEST_TIMEOUT	20 seconds	HTTP timeout for asset fetches
ASSET_PREFETCH_WORKERS	12	Parallel download threads
DEFAULT_MAX_PAGES	300 (deep) / 60 (fast)	Crawl page limit
DEFAULT_MAX_DEPTH	8 (deep) / 4 (fast)	Crawl depth limit
TRACKING_QUERY_PREFIXES	utm_, fbclid, gclid, mc_	Stripped from URLs

6. Module 2: Smart Dynamic Conversion

`File: smart_conversion.py (~69 KB, ~1700 lines)`

This module transforms a static cloned website into a **data-driven, editable CMS**. It analyzes the HTML, detects the website type, extracts content into structured models, generates a database schema, CRUD API routes, admin panel spec, and injects a JavaScript bridge for live data binding. **No AI/ML is used** - it is entirely heuristic-based (regex, pattern matching, keyword spotting).

6.1 Conversion Pipeline (12 Steps)

`File: smart_conversion.py : build_conversion_blueprint()`

1. Parse HTML

BeautifulSoup('html.parser') parses the cloned HTML into a DOM tree.

2. Detect Site Type

detect_site_type() classifies into 8 types: ecommerce, restaurant, portfolio, blog, education, saas, real_estate, agency. Uses keyword frequency in visible text, form detection, price pattern matching.

3. Extract Sections

extract_sections() identifies 24+ semantic sections (hero, banner, gallery, features, testimonials, footer, etc.) using tag names, class/id patterns, and ARIA roles. Each section gets a CSS selector and unique section_key.

4. Extract Media

extract_media_assets() scans all <img>, <video>, <audio>, <source>, CSS url(), <svg> with usage tracking. Builds asset_key, media_type, section assignment.

5. Localize Media

materialize_media_asset() downloads each asset with SHA-1 hashing. localize_clone_html() replaces original URLs with local paths in the HTML.

6. Extract Navigation

extract_navigation() pulls up to 30 nav links with CSS selectors and section context.

7. Extract Content

extract_text_blocks() pulls headings/paragraphs. extract_price_tokens() finds currency patterns. extract_forms() identifies form fields.

8. Infer Data Models

extract_content_models() generates models based on site type. infer_fields_for_model() provides 25+ predefined schemas (products, courses, posts, etc.).

9. Generate Seed Records

extract_seed_records() creates initial records by mapping extracted text, prices, and media to model fields. build_record_from_source() handles the mapping.

10. Build API Routes

build_backend_routes() generates CRUD endpoints for every model: GET list, GET by id, POST create, PATCH update, DELETE remove.

11. Build Admin Spec

build_admin_sections() creates admin panel section definitions with columns, actions, and display config.

12. Calculate Confidence

calculate_confidence_breakdown() scores extraction quality across 9 components: DOM depth, text coverage, section count, style extraction, layout quality, media recovery, link coverage, interactivity, type detection.

6.2 Site Type Detection

`File: smart_conversion.py : detect_site_type()`

Site Type	Detection Signals
ecommerce	Keywords: cart, checkout, add to cart, price, buy. Price patterns. Product forms.
restaurant	Keywords: menu, reservation, cuisine, chef, dine. Booking forms.
portfolio	Keywords: portfolio, projects, works, gallery, showcase.
blog	Keywords: blog, article, post, author, published, read more.
education	Keywords: course, enroll, student, faculty, semester, campus.
saas	Keywords: pricing, features, plans, subscribe, API, dashboard.
real_estate	Keywords: property, listing, bedroom, sqft, agent, rent.
agency	Keywords: services, clients, case study, team, solutions.

6.3 Confidence Scoring (9 Components)

`File: smart_conversion.py : calculate_confidence_breakdown()`

Component	What It Measures	How
DOM Depth	HTML structure complexity	Counts nested elements, min threshold
Text Coverage	Extractable text volume	Word count vs minimum threshold
Sections	Identifiable page regions	Count of detected semantic sections
Styles	CSS extraction success	Colors, fonts, logos found
Layout	Page structure quality	Header/nav/main/footer presence
Media Recovery	Asset download success rate	Downloaded / total assets ratio
Links	Navigation completeness	Internal links found and resolved
Interactivity	Interactive elements found	Forms, buttons, dropdowns count
Type Detection	Classification confidence	Signal strength for detected type

6.4 JavaScript Clone Bridge

`File: smart_conversion.py : inject_clone_bridge()`

A <script> block is injected into the cloned HTML that runs on DOMContentLoaded. It fetches data from the API and updates the DOM live:

- `/api/public/{site_id}/site_sections` - Updates section titles, summaries, and media via CSS selectors
- `/api/public/{site_id}/navigation_items` - Updates nav links
- `/api/public/{site_id}/theme_styles` - Applies CSS custom properties (colors, fonts)
- `/api/public/{site_id}/site_pages` - Updates page metadata

7. Module 3: Main Application Orchestrator

File: app.py (~99 KB, ~2400 lines)

This is the **main Streamlit application** - the central orchestrator that ties all modules together. It provides the UI, manages user sessions, and coordinates the cloning, conversion, editing, and export workflows.

7.1 Streamlit Menu Pages

Menu Item	What It Does	Modules Used
Clone New Site	Enter URL, select mode (Deep/Fast), clone website, preview, save to DB	clone_engine
Manage My Sites	List sites, live HTML editor with preview, commit changes, download ZIP	None (direct DB)
Smart Dynamic Conversion	Select theme, generate blueprint, view architecture, build export	smart_conversion, connected_export
Drag & Drop Builder	Visual page editor via GrapesJS in iframe	dragdrop_flask (subprocess)

7.2 Cloning Workflow (Clone New Site)

File: app.py : scrape_website(), build_clone_progress_callback()

User selects Deep (300 pages, depth 8) or Fast (60 pages, depth 4) mode from CLONE_MODE_PRESETS. Calls `clone_site_bundle()` with a progress callback that updates a Streamlit progress bar through 4 phases: crawl (0-50%), prefetch (50-72%), rewrite (72-96%), archive (96-100%). Result is stored in session_state. User clicks 'SAVE TO PROJECT DATABASE' to persist.

7.3 Conversion Workflow (Smart Dynamic Conversion)

File: app.py : generate_clone_runtime()

The most complex workflow. Steps: (1) User selects a saved site and an admin theme (5 options: fabricator-current, saas-minimal, glass-premium, enterprise-command, creative-studio). (2) Calls `build_conversion_blueprint()`. (3) `load_saved_bundle_snapshot()` reads cloned pages/assets from disk. (4) `apply_bundle_runtime_context()` merges cloned data into blueprint: processes navigation items, merges media assets via `merge_clone_media_assets()` with identity-based deduplication, builds page records with SEO metadata. (5) `create_scaffold_files()` writes JSON/Python files. (6) Saves to DB: blueprint, spec, models, records, media. (7) Displays results in 4 tabs: Architecture, Admin App, API+Files, Pipeline.

7.4 Media Asset Merge Algorithm

File: app.py : merge_clone_media_assets(), media_identity_tokens()

Merges blueprint media (user uploads) with bundle media (cloned assets) using identity tokens. `media_identity_tokens()` generates tokens: [hash, asset_key, original_url, file_url, local_path]. For each bundle asset, checks if any token matches an existing asset. If match: merge metadata and usage info, preserve custom uploads. If no match: append as new asset. This ensures no duplicates while preserving all sources.

8. Module 4: Drag & Drop Visual Editor

File: dragdrop_flask.py (~50 KB, ~1280 lines)

A Flask application serving on port 5001 that provides: (a) A GrapesJS visual page editor, (b) Full CRUD admin API for dynamic content, (c) Media upload/management, (d) Clone preview serving. Embedded in the main Streamlit app as an iframe.

8.1 GrapesJS Editor

File: dragdrop_flask.py : EDITOR_TEMPLATE, /editor/

The editor loads via `GET /editor/<site_id>`. HTML template embeds GrapesJS from CDN with Bootstrap 4 blocks plugin. Layout: 240px left sidebar (blocks + layers) | flexible canvas | 260px right sidebar (styles + traits). User can drag components, edit text, change styles, switch device preview (Desktop/Tablet/Mobile at 375x812). Save posts HTML+CSS to `POST /save_site/<site_id>` which stores to DB and updates the bundle archive.

8.2 Editor Save Flow

```
User clicks Save -> JS extracts editor.getHtml() + editor.getCss()
-> POST /save_site/{site_id} with {html, css} body
-> Server: UPDATE cloned_sites SET html_content = ? WHERE id = ?
-> Server: update_saved_bundle_entry() writes to disk bundle
-> Response: {ok: true}
```

8.3 Clone Preview Serving

File: dragdrop_flask.py : /clone//, /clone//

Serves the cloned website for live preview. `GET /clone/{site_id}/` returns the entry HTML (from bundle or DB). `GET /clone/{site_id}/{path}` serves specific assets or sub-pages from `project_assets/clone_packages/site_{id}/site/`.

8.4 Dynamic Content API

Endpoint	Method	Purpose
/api/admin/login	POST	Authenticate admin user, return token
/api/admin/me	GET	Get current user from token
/api/projects	GET	List user's projects with record counts
/api/projects/{id}/spec	GET/PATCH	Read/update SiteSpec and bindings
/api/admin/{sid}/{resource}	GET/POST	List or create records for a resource
/api/admin/{sid}/{resource}/{id}	PATCH/DEL	Update or delete a specific record
/api/admin/{sid}/{resource}/{id}/publish	PATCH	Toggle draft/published status
/api/admin/{sid}/media	GET/POST	List media or upload new files
/api/admin/{sid}/media/{id}	PATCH/DEL	Edit metadata or delete media
/api/public/{sid}/{resource}	GET	Public API - only published records (no auth)
/dynamic-api/{sid}/meta	GET	Collection metadata for a site
/dynamic-api/{sid}/{model}	GET/POST	Dynamic CRUD for any model

9. Module 5: Connected Export Packager

File: connected_export.py (~50 KB, ~1200 lines)

Packages everything into a **self-contained, deployable system** - a ZIP file with its own Flask backend, SQLite database, admin panel, and cloned website. Runs standalone on port 5010.

9.1 Export Algorithm

File: connected_export.py : build_connected_export()

- 1. Validate prerequisites: admin build exists, clone saved, dynamic artifacts generated
- 2. Load context: site metadata, user credentials, SiteSpec from DB
- 3. Create directory structure: cloned_website/, admin_system/
- 4. Copy cloned assets via copytree_clean() to cloned_website/
- 5. sync_export_clone_html(): Strip preview paths, inject clone bridge into ALL HTML files
- 6. Copy admin panel build (React dist) to admin_system/admin_panel/
- 7. export_site_database(): Filter DB to user-owned records only, write to site.db
- 8. write_backend_runtime(): Generate Flask app.py from 4 template segments
- 9. write_export_docs(): Create README.md, export_manifest.json, startup scripts
- 10. ZIP everything with shutil.make_archive()

9.2 Export Package Structure

```
{site_name}_connected_system/
  cloned_website/          # Static HTML/CSS/JS/assets
    index.html
    assets/css/, js/, images/, fonts/
    _fabricator/manifest.json
  admin_system/
    admin_panel/           # React SPA build
    backend/
      app.py                # Generated Flask server
      requirements.txt      # flask, flask-cors, itsdangerous
      start_backend.bat
    database/site.db       # Standalone SQLite
    generated_dynamic/     # Schema, spec, media manifest JSONs
    media/site_{id}/       # Cloned media files
  README.md
  export_manifest.json
  start_connected_system.bat
```

9.3 Generated Backend Routes

The generated `app.py` is built from 4 template segments (BACKEND_TEMPLATE_HEAD through BACKEND_TEMPLATE_TAIL_4). It includes: clone website serving at `/site/`, admin panel at `/admin/`, full CRUD API at `/api/admin/{sid}/{resource}`, media upload at `/api/admin/{sid}/media`, and public read API at `/api/public/{sid}/{resource}`. Auth uses the same URLSafeSerializer token system.

10. Module 6: Admin Panel Frontend

`File: admin/src/ (React 19 + TypeScript + Vite)`

A modern React SPA that provides the CMS admin interface. Built with React 19, TypeScript 5.9, React Router 7 for routing, and Vite 8 for bundling. Served from `/admin/` path.

10.1 Route Structure

Route	Component	Purpose
/admin/login	LoginPage	Username/password auth form
/admin/site/:projectId	AdminShell	Main layout with sidebar + content area
/admin/site/:projectId/ (index)	ProjectDetailPage	Overview: metrics, resources, diagnostics
/admin/site/:projectId/resources/:key	ResourceListPage	Table of records with search/filter/publish
/admin/site/:projectId/resources/:key/new	RecordEditorPage	Create new record (form from schema)
/admin/site/:projectId/resources/:key/:id	RecordEditorPage	Edit existing record
/admin/site/:projectId/media	MediaPage	Media library grid with preview/upload

10.2 Component Architecture

- **AdminShell** (admin/src/AdminShell.tsx): Two-panel layout. Left sidebar with project card, nav sections (Workspace, Content, Structure), resource counts. Top bar with theme/mode toggle. Fetches ProjectSpecResponse on mount.
- **LoginPage** (admin/src/LoginPage.tsx): Posts to /api/admin/login. Stores token + user in localStorage. Redirects to /site/:projectId.
- **ResourceListPage** (admin/src/ResourceListPage.tsx): Data table from schema. Search, status filter (all/draft/published). Row actions: Edit, Publish, Delete. Metric cards.
- **RecordEditorPage** (admin/src/RecordEditorPage.tsx): Dynamic form from SiteSpec field schema. Field types: text->textarea, number->input, boolean->select, media->URL+picker dropdown. Auto-slugifies title fields.
- **MediaPage** (admin/src/MediaPage.tsx): Grid view with type filtering (image/svg/icon/video/audio). Preview panel. Upload, replace, delete actions.

10.3 Theming System

`File: admin/src/lib/themes.ts`

5 built-in themes with light/dark modes each. Applied via CSS custom properties (`--accent, --border, --text-h`) on HTML data-theme + data-mode attributes. Theme choice persisted per-project in localStorage.

Theme ID	Name	Default Mode
fabricator-current	Neon Depth Premium CMS	Dark
saas-minimal	Clean SaaS Dashboard	Light
glass-premium	Commerce Operations	Dark
enterprise-command	Executive Workspace	Light
creative-studio	Campus Education	Dark

11. End-to-End User Journey Flow

This section traces a complete user journey from registration to a deployed standalone system.

Complete user journey: Register -> Clone -> Save -> Convert -> Edit -> Manage -> Export -> Deploy. Each step shows the module responsible and the database/file operations performed.

12. Complete API Endpoints Reference

`File: dragdrop_flask.py (development) | connected_export.py (generated production)`

12.1 Authentication Endpoints

Method	Path	Auth	Request	Response
POST	/api/admin/login	None	{username, password}	{token, user: {id, username}}
GET	/api/admin/me	Bearer	-	{id, username}

12.2 Project Endpoints

Method	Path	Purpose
GET	/api/projects	List all projects with record counts per resource
GET	/api/projects/{id}/spec	Get full SiteSpec + bindings + resources
PATCH	/api/projects/{id}/spec	Update SiteSpec bindings

12.3 Content CRUD Endpoints

Method	Path	Purpose
GET	/api/admin/{sid}/{resource}	List all records (with ?q= search, ?status= filter)
POST	/api/admin/{sid}/{resource}	Create new record with auto-slug
PATCH	/api/admin/{sid}/{resource}/{id}	Update record fields + regenerate slug
DELETE	/api/admin/{sid}/{resource}/{id}	Delete record permanently
PATCH	/api/admin/{sid}/{resource}/{id}/publish	Toggle status: draft <-> published

12.4 Media Endpoints

Method	Path	Purpose
GET	/api/admin/{sid}/media	List media (with ?q= search, ?type= filter)
POST	/api/admin/{sid}/media	Upload file (multipart/form-data) with SHA-1 hash
PATCH	/api/admin/{sid}/media/{id}	Update alt_text, section_key, or replace file
DELETE	/api/admin/{sid}/media/{id}	Soft-delete (status='deleted')
GET	/media/{sid}/{filename}	Serve media file directly

12.5 Public Read-Only API

Method	Path	Auth	Purpose
GET	/api/public/{sid}/{resource}	None	Fetch published records only. Used by clone bridge JS.

13. Key Algorithms Deep Dive

13.1 BFS Web Crawling

File: cloner_engine.py : crawl_site_pages()

Algorithm: Breadth-First Search using `collections.deque`. Starts with root URL at depth 0. For each page: fetch HTML, extract all internal links, add unseen links to queue with depth+1. Also processes sitemaps from robots.txt as a secondary queue. Terminates when queue is empty OR page count reaches max_pages OR all pages at max_depth are processed.

```
queue = deque([(root_url, 0)])  # (url, depth)
visited = set()
while queue and len(pages) < max_pages:
  url, depth = queue.popleft()
  if url in visited or depth > max_depth: continue
  visited.add(url)
  html = fetch_html_page(url)
  links = collect_internal_navigation_urls(html, base_url)
  for link in links:
    if link not in visited: queue.append((link, depth + 1))
```

13.2 Asset Deduplication (SHA-1 Hash Index)

File: cloner_engine.py : clone_binary_asset(), clone_stylesheet_asset()

Every downloaded asset is hashed with SHA-1. A hash index maps hash -> output path. Before writing a file, the engine checks if the hash already exists. If yes, the new reference points to the existing file (no duplicate write). The manifest marks these as 'deduplicated'. This significantly reduces bundle size when sites reuse the same image across pages.

13.3 CSS URL Rewriting

File: cloner_engine.py : rewrite_css_text()

Processes all `@import url(...)` and `url(...)` patterns in CSS. For each URL: resolves relative to CSS file location, checks if the asset was downloaded, and replaces with a relative path from the CSS output location to the asset output location. Uses `posixpath.relpath()` for correct relative path calculation.

13.4 Media Identity Merge

File: app.py : merge_clone_media_assets(), media_identity_tokens()

When merging blueprint media with bundle media, each asset generates identity tokens: [hash, asset_key, original_url, file_url, local_path]. Two assets are considered identical if ANY token matches. This handles the case where the same image has different URLs (CDN vs local) or different filenames.

13.5 Site Type Classification (Heuristic)

File: smart_conversion.py : detect_site_type()

Extracts visible text from HTML, converts to lowercase, counts keyword occurrences for each of 8 categories. Also checks: form presence (contact/order/reservation), price patterns (RS, PKR, $, numbers), domain name signals. The type with highest weighted score wins. Confidence = weighted score / max possible.

13.6 Section Extraction (DOM Walking)

File: smart_conversion.py : extract_sections()

Walks the DOM looking for semantic markers: `<header>`, `<nav>`, `<main>`, `<footer>`, `<section>`, `<article>`, plus divs with class/id containing keywords like 'hero', 'banner', 'features', 'testimonials', 'gallery', 'pricing', 'contact'. 24+ section types are recognized. Each gets a CSS selector via `build_element_selector()` and a unique section_key via `build_section_key()`.

13.7 Parallel Asset Prefetching

File: cloner_engine.py : prefetch_assets()

Uses `concurrent.futures.ThreadPoolExecutor(max_workers=12)` to download all discovered assets in parallel before the rewriting phase. Results cached in `prefetched_payloads` dict. During rewriting, assets are read from cache instead of making additional HTTP requests. This dramatically speeds up cloning of asset-heavy sites.

14. Diagram Explanations

This section provides plain-English explanations for every diagram in this document.

14.1 Architecture Diagram (Section 1)

What it shows: The system has 4 layers. The **User Browser** at the top connects to **Streamlit UI** (app.py on port 8501) which handles the main interface. Streamlit communicates with **Flask Backend** (dragdrop_flask.py on port 5001) which provides the visual editor and API. Both read/write to the **SQLite Database** (users.db with 8 tables). The connected_export module packages everything into a **Standalone Package** that runs independently on port 5010. Arrows show the direction of data flow between layers.

14.2 Auth Flow Diagram (Section 3)

What it shows: The complete login process step-by-step. User submits credentials -> Server hashes password with SHA-256 -> Queries database for match -> If found, generates a token using URLSafeSerializer -> Returns token to client -> Client stores in localStorage -> Every subsequent API call includes the token in the Authorization header -> Server validates token before processing any admin request. This is a **stateless token-based auth** system (no server-side sessions).

14.3 User Journey Diagram (Section 11)

What it shows: The 10-step lifecycle of using the application from start to finish. Steps 1-2: Registration and URL input. Steps 3-4: The cloning engine crawls the website and saves results. Steps 5-6: Smart conversion analyzes the clone and generates database models. Steps 7-8: User refines using the visual editor and admin panel. Steps 9-10: Everything is packaged into a standalone ZIP and deployed. Each step lists the exact Python module and database operation involved, so a jury member can trace any step back to specific code.

14.4 Module Dependency Table (Section 1)

What it shows: Which module imports from which other module, and the specific functions used. app.py is the central hub that imports from all three utility modules. dragdrop_flask.py and connected_export.py both import from cloner_engine (for bundle management) and smart_conversion (for the clone bridge injection). No circular dependencies exist - the dependency graph is a clean DAG (Directed Acyclic Graph).

14.5 Database Schema Tables (Section 4)

What they show: Every column in all 8 database tables with types, constraints, and purposes. The **users** table stores credentials. **cloned_sites** stores the raw cloned HTML. **conversion_blueprints** stores the analysis result. **site_specs** stores the frontend binding config. **dynamic_models** defines content schemas (like a CMS content type). **dynamic_records** holds actual content data (like CMS entries). **media_assets** tracks all images/videos/files. Foreign keys (user_id, site_id) ensure each user only accesses their own data.

14.6 API Endpoints Tables (Section 12)

What they show: Every HTTP endpoint in the system organized by category. Auth endpoints handle login/token. Project endpoints manage site metadata. Content CRUD endpoints allow creating, reading, updating, deleting records for any dynamic model. Media endpoints handle file uploads with deduplication. Public endpoints serve published-only data without authentication (used by the clone bridge script to update the cloned website's DOM with fresh content from the database).

End of Document

This document covers every module, algorithm, database table, API endpoint, and data flow in the AI Website Fabricator project. All file references point to exact source files in the project directory. For any jury question, cross-reference the relevant section number with the file path provided.